AOSP 进阶丛书 · 第十三册 · ANDROID 13 (API 33)

SystemUI 通知体系

SystemUI 的身份（特权 App 而非系统服务）、CoreStartable 微服务模型、通知从 notify() 到状态栏的完整管线（NMS 排序/通道/速率/Bubbles）、最近任务截图与 Launcher3 状态驱动架构。

源码基线：AOSP Android 13 (TP1A · SDK 33) · 行号全部实测核验

目 录

前言：用户看见的"系统" 2
第一章 SystemUI：不是服务，是特权应用 2
第二章 通知管线：notify() 到显示的完整链 3
第三章 最近任务与快照 5
第四章 Launcher3：桌面与图标 5
第五章 设计决策八问 5
附录 A 观测实操 6
附录 B 快问快答（14 条） 6
附录 C 自检题 7
教科书习题集（分级） 7
术语表 7
排障手册：SystemUI 与通知问题定位 7

本册速览

SystemUI 是普通 App：SystemUIApplication:62 + CoreStartable 注册制
通知管线：notify()→NMS:349 排序/通道/速率→SystemUI 渲染
最近任务：TaskSnapshotController 截图+RecentTasks 重组
Launcher3：StatefulActivity 状态驱动 · LauncherModel 监听装/卸
设计决策：为什么 SystemUI 进程独立/通知为什么有通道/快照为什么在 system_server

前言：用户看见的"系统"

前十二册讲完了系统内部的机器；本册讲**用户看见的那个"系统"**：状态栏/通知/快捷设置/锁屏/最近任务——它们全部住在一个名叫 SystemUI 的**特权应用**里，加上作为桌面的 Launcher3，共同构成"用户体验的门面"。

体例不变：真实源码锚点 + 逐段注释 + 流程图 + 设计决策。本册主角：`frameworks/base/packages/SystemUI/`（SystemUI 主体）、`frameworks/base/services/core/.../notification/`（NMS）、`packages/apps/Launcher3/`（桌面）。

第一章 SystemUI：不是服务，是特权应用



图 1 SystemUI 的身份与出生（SystemUIApplication:62）、CoreStartable 微服务模型、主要组件清单与四条独立进程理由

1.1 身份逐段（真实锚点）

java frameworks/base/packages/SystemUI/…（骨架+讲解注释）

```
// SystemUIApplication.java:62
public class SystemUIApplication extends Application
implements Configuration.ProviderService {
// 讲解注释①：SystemUI 是一个普通 App——有 Application/有 Activity/有 Service
// 进程名 com.android.systemui，uid=SYSTEM（platform 签名特权）
```

```
// 讲解注释②：启动方式：SystemServer.java:3053 startSystemUi()
// → startService(SystemUIService) → Application.onCreate()
// → 依次启动各 CoreStartable（组件）
}
// CoreStartable.java:34
public abstract class CoreStartable implements Dumpable {
// 讲解注释③：SystemUI 内部的"微服务注册制"：
// StatusBar / Recents / QS / VolumeUI… 每个实现此类
// → Dependency.java 注入（ServiceLocator 模式）
// → 每个组件独立 onStart()，可 dumpsys 分组件查看
}
```

先修要求与学习路线

先修：已读第一册（架构总览）。

读完本书你将能够：

- 解释SystemUI 与通知的核心机制与设计决策
- 追踪关键调用链并标注每次跨进程跳
- 使用 dumpsys/日志定位本领域问题
- 回答书7 对应章节的面试题

设计洞察 · 逐段精读：为什么 SystemUI 是独立 App 而非 system_server 线程

四条理由：**崩溃隔离**（UI 死了系统活着可自动重启）、**渲染独立**（UI 帧率不拖服务线程）、**独立 OTA**（Mainline 可单独升级 SystemUI 组件）、**内存配额独立**。面试高频"SystemUI 崩了会怎样"：init 检测 critical → 自动重启（闪一下状态栏恢复）。

第二章 通知管线：notify() 到显示的完整链



图 2 通知四段：App 通知通道→NMS 排序策略→SystemUI 渲染，含 Bubbles/速率/桶联动

2.1 NMS 核心逐段（真实锚点）

java NotificationManagerService.java（骨架+讲解注释）

```
// :349 public class NotificationManagerService extends SystemService
// ——SystemServer startOtherServices 启动（书2 §七）
//
// enqueueNotificationInternal(:6397/6404)
// 讲解注释①：权限/通道/速率三道闸：
// • 通道不存在 → dropped（8+ 必须先 createNotificationChannel）
// • 速率超限（mRateLimitExceeded）→ dropped
// • Standby RESTRICTED 桶（书9 §五）→ 静默
// 讲解注释②：NotificationRecord 入 mNotificationList（按排名排序）
// 排序由 NotificationSignalExtractor 链决定：
// Importance→Channel→ZenMode→Visibility→…各提取器逐级调整排名
// 讲解注释③：buzzBeepBlink——声音/振动/LED 三个通道策略在此
// 讲解注释④：notifyPostedListeners→SystemUI 的 NotificationListener
// binder 回调 → SystemUI 渲染面板
```

设计洞察 • 排障树："通知不显示"的排查序

通道开关→勿扰模式→importance 被降→速率限制→后台桶限制。dumpsys notification 一次看全这五项。面试高频："为什么同样代码有的设备响有的不响"——厂商策略差异+勿扰默认值差异。

第三章 最近任务与快照

TaskSnapshotController（wm/）在 Activity 切后台时截图（经 SurfaceFlinger 的 screenshot 接口→GraphicBuffer 存储）；RecentTasks（:109）管理任务栈重组（分屏/最近列表排序/重新打开）。**Launcher3 的 RecentsView 消费这两者**：截图显示+TaskKey 定位→点按→ATMS.startActivityFromRecents（书4 §一 的入口之一）。

`dumpsys activity recents` 看当前任务栈全表；`dumpsys window | grep snapshot` 看快照状态。快照的意义：**最近任务"秒开"的视觉假象**——先显示截图再让 App 追上真实帧。

第四章 Launcher3：桌面与图标

java packages/apps/Launcher3（骨架+讲解注释）

```
// Launcher.java:238
public class Launcher extends StatefulActivity<LauncherState>
// 讲解注释①：StatefulActivity=状态驱动 UI——NORMAL/APPLICATION/
// OVERVIEW 等状态各自定义可交互元素，切换即重绘
// LauncherModel.java:87
public class LauncherModel extends LauncherApps.Callback
implements InstallSessionTracker.Callback {
// 讲解注释②：模型层——监听 App 安装/卸载（LauncherApps 系统服务
// ↔ 书6 PMS 的安装事件），维护图标与 widget 布局数据库
// 讲解注释③：IconCache——App 图标/标签缓存（跨启动持久化）
}
```

Launcher3 的预测（13）：`packages/apps/Launcher3/src/.../predictor/` 基于使用统计的 App 预测排序（与书9 UsageStats 联动）。

第五章 设计决策八问

- **为什么 SystemUI 是 App 不是系统服务？** 崩溃隔离+渲染独立+独立 OTA+内存独立（第一章四条）。
- **为什么通知要走 NMS 而非直接给 SystemUI？** 排序/策略/速率/通道检查需要全局视角（权限数据在 system_server）——SystemUI 只是渲染者。
- **为什么 8+ 强制通知通道？** 用户需要细粒度控制（"关广告不关聊天"）；通道是"通知偏好组"的载体。
- **为什么最近任务用截图而非实时窗口？** 性能（截图是静态 GraphicBuffer）+隐私（后台 App 可能已变内容）；"秒开"是假象先给截图再让 App 追帧。
- **为什么 Launcher 用 StatefulActivity？** 桌面 UI 状态多（NORMAL/OVERVIEW/WIDGET_PICKER…）且切换频繁——状态驱动让每种状态的交互规则声明式定义。

- **为什么 LauncherModel 而非直接操作数据库？** 装卸事件异步到达（Binder 回调线程）→需统一调度主线程刷新→Model 是"事件→状态→UI"的中间层。
- **为什么通知排序是提取器链而非单一规则？** 排序因素多维（通道重要度×时效×用户偏好×勿扰）——链式提取器可按序调整且可插拔。
- **为什么 SystemUI 逐步 Dagger 化？** Dependency.get() 全局单例测试难；Dagger 编译期注入+可测试替换——工程可维护性的渐进改造。

附录 A 观测实操

目标	命令	看点
通知全表	dumpsys notification	排序/通道/速率/勿扰
状态栏	dumpsys statusbar	面板/通知行/可见性
最近任务	dumpsys activity recents	任务栈/快照
SystemUI 组件	dumpsys activity service SystemUI	CoreStartable 状态
Launcher	dumpsys activity launcher	图标/widget 布局

附录 B 快问快答（14 条）

- SystemUI 是系统服务吗？——不是，特权 App（uid SYSTEM、platform 签名）。
- SystemUI 崩了怎样？——init critical 自动重启。
- 通知不显示第一查？——通道开关（8+ createChannel 是前提）。
- 排序由谁决定？——NotificationSignalExtractor 链。
- Bubbles 是什么？——通知的浮动形态（对话类）。
- 通知速率限制在哪？——NMS enqueueNotificationInternal（:6397）。
- RESTRICTED 桶的通知？——被静默（书9 联动）。
- 最近任务截图谁截的？——TaskSnapshotController→SF screenshot。
- Launcher 什么架构？——StatefulActivity 状态驱动。
- 图标缓存在哪？——LauncherModel 的 IconCache（持久化）。
- Launcher3 怎么知道装了新 App？——LauncherApps.Callback（书6 PMS 事件）。
- 通知点击怎么打开 Activity？——书4 PendingIntent 代执行。
- SystemUI 可独立升级吗？——部分组件可（Mainline）。
- 学完本册？——书14 HAL 实战（从应用到硬件驱动的最后一跳）。

附录 C 自检题

- SystemUI 的四条独立进程理由。
- 通知五步链（通道→NMS 排序→速率→通知监听→渲染），标出 binder 跳点。
- 最近任务截图从截到显示的链路。
- 为什么通知通道是 8+ 硬要求？

教科书习题集（分级）

第一级：简答题（检验概念理解）

（从附录 B 快问快答中挑 5 题，闭卷手写答案。）

第二级：分析题（检验机制理解）

- **追踪调用链**：画出本册核心机制的完整调用图。
- **失败推演**：核心机制某步失败，系统会怎样？
- **跨册串联**：本册机制与 Binder 十跳（书3）的交汇点。

第三级：设计题

- **架构演进**：重构本册核心机制，你会怎么改？
- **排障方案**：设计分层定位方案。

术语表

（术语表：从附录 B 提取本册专属术语，按拼音排序。）

排障手册：SystemUI 与通知问题定位

1. dumpsys 定位状态 → 2. logcat 查看日志 → 3. 深入各章。

深度实战：SystemUI 与通知的排障场景分析

场景：通知图标不消失（幽灵通知）

1. dumpsys notification → 确认该通知在 mNotificationList

2. App 调了 cancel 但 SystemUI 没收到？→ 检查 notificationlistener 绑定

3. ZenMode 拦截→ 检查 mZenMode

4. SELinux 拦截 NMS→SystemUI 的 binder 调用 → avc denied

5. 修复：force-stop 对应 App → 通知自动清除